

尚硅谷大模型项目之掌柜问数

1. 项目概述

掌柜问数是一个基于自然语言处理与数据分析技术的智能数据服务系统，面向数据仓库应用场景，旨在帮助用户通过对话方式高效获取数据仓库中的数据洞察。用户无需掌握复杂的查询语法，即可用自然语言提出问题，系统自动完成对数据仓库数据的理解、计算分析与结果可视化，大幅提升数据使用效率，降低数据分析门槛，助力业务决策智能化。

统计每个月销量最高的商品

🤖

- 抽取关键词
- 召回字段信息
- 召回指标信息
- 召回字段值
- 合并召回信息
- 筛选指标信息
- 筛选表信息
- 添加上下文信息
- 生成SQL
- 验证SQL语句
- 执行SQL语句

🤖

year	month	product_name	total_sales
2025	1	奥利奥巧克力夹心饼干	47
2025	2	乐事原味薯片 150g	54
2025	3	蒙牛纯牛奶 250ml*12	118

请输入你的问题...

发送

2. 项目架构

本项目以数据仓库的元数据为核心，使用 MySQL 存储结构化元数据信息，结合 Qdrant 构建语义向量索引、Elasticsearch 构建全文索引，形成统一的元数据知识库。查询过程中，系统首先根据用户自然语言问题进行多路召回，筛选相关表、字段及指标定义，再将元数据信息与用户问题共同输入大模型生成 SQL，最终完成自动查询与结果返回，确保生成结果的准确性与可控性。

2.1 元数据知识库

元数据知识库作为数据仓库的语义基础设施，用于集中管理和高效检索表结构、字段定义、字段取值示例及复杂指标说明等元数据信息，支撑后续的智能检索与 SQL 生成。

元数据信息主要来源于两部分：一部分数据仓库自动采集，另一部分由人工进行补充与配置。完整的元数据统一存储于 MySQL 数据库中，并对其部分关键信息构建向量索引和全文索引，以提升语义召回与关键词召回的效果。

2.2.1 元数据库

元数据库共包含四张表，具体结构如下图所示：

metric_info

🔑 id: varchar(64)

name: varchar(128)

description: text

relevant_columns: json

alias: json

指标编码

指标名称

指标描述

关联...

指标别名

column_metric

🔑 column_id: varchar(64)

🔑 metric_id: varchar(64)

列编号

指标编号

column_info

🔑 id: varchar(64)

name: varchar(128)

type: varchar(64)

role: varchar(32)

examples: json

description: text

alias: json

table_id: varchar(64)

列编号

列名称

数据类型

列类型(prima...

数据示例

列描述

列别名

所属表编号

table_info

🔑 id: varchar(64)

name: varchar(128)

role: varchar(32)

description: text

表编号

表名称

表类型(fact/d...

表描述

2.2.2 向量索引

本项目选用 [Qdrant](#) 作为向量数据库，向量索引主要用于对**指标信息**和**字段信息**进行语义召回。向量索引的构建内容具体如下：

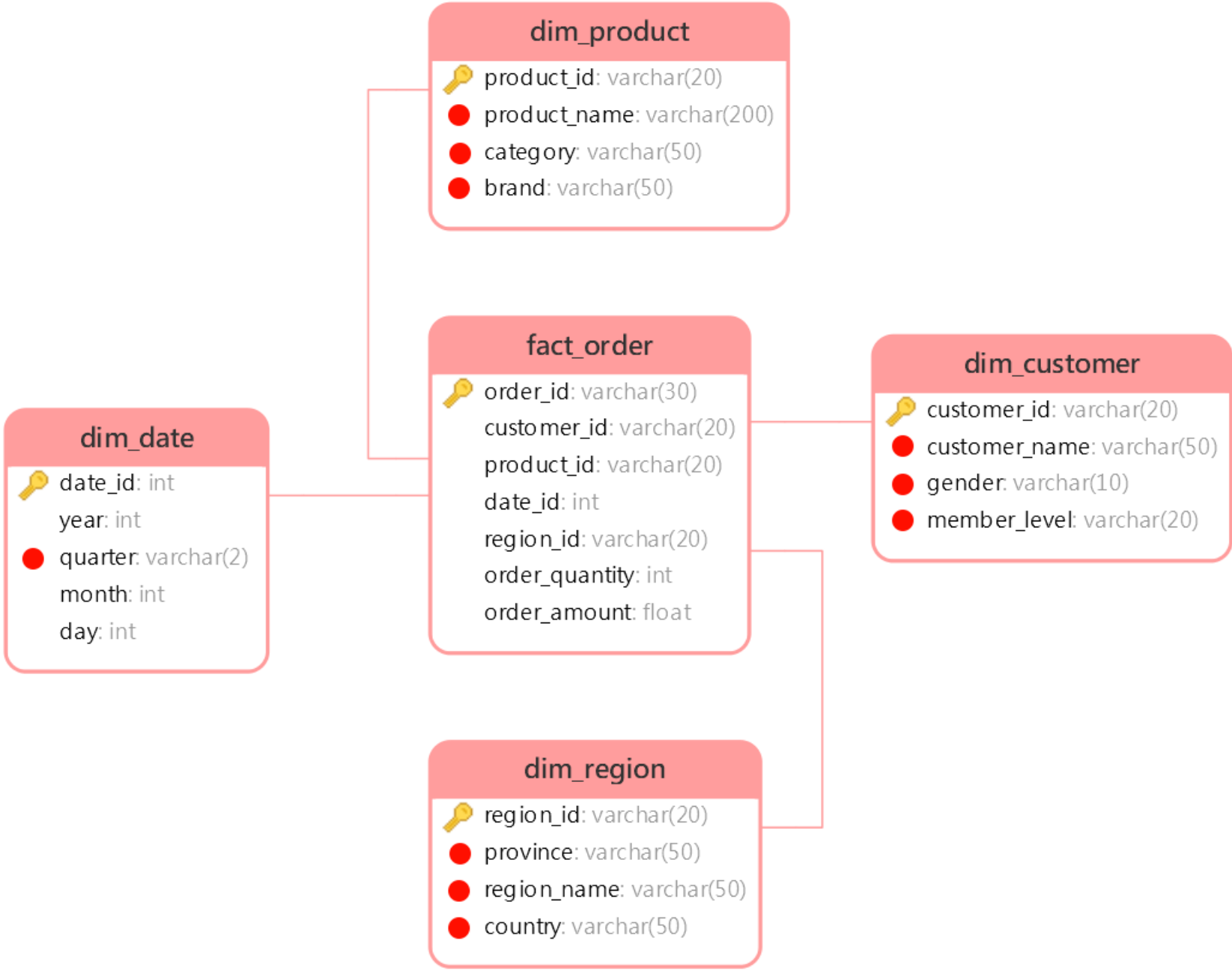
- metric_info

metric_info	
🔑 id: varchar(64)	指标编码
● name: varchar(128)	指标名称
● description: text	指标描述
relevant_columns: json	关联...
● alias: json	指标别名

- column_info

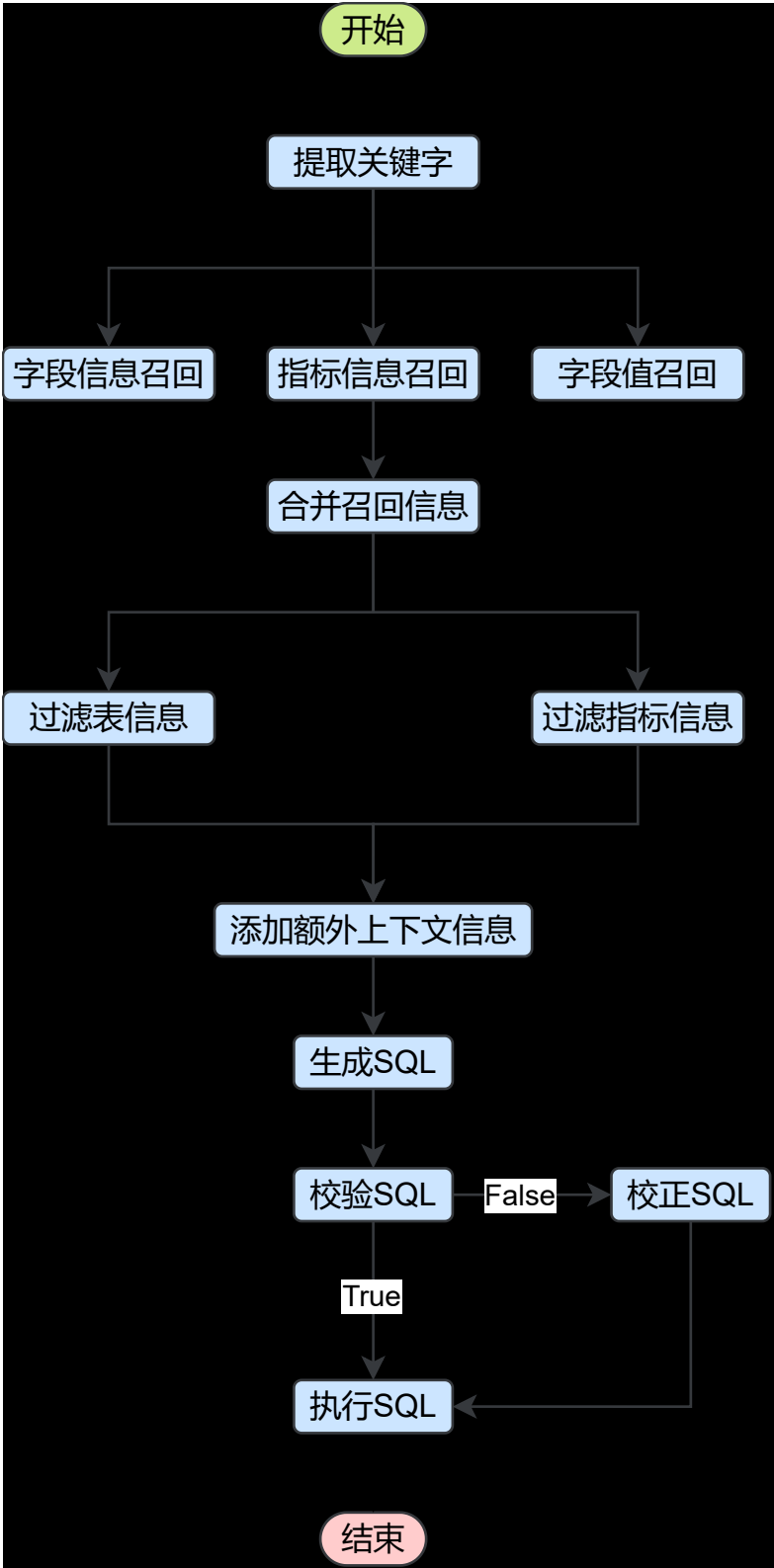
2.2.4 全文索引

本项目使用 Elasticsearch 作为全文检索引擎，全文索引主要用于对**字段取值**进行检索与匹配，索引内容以各类维度值为主，具体如下：



2.2 问数智能体

本项目的智能体主体基于Langgraph构建，具体结构如下图所示：



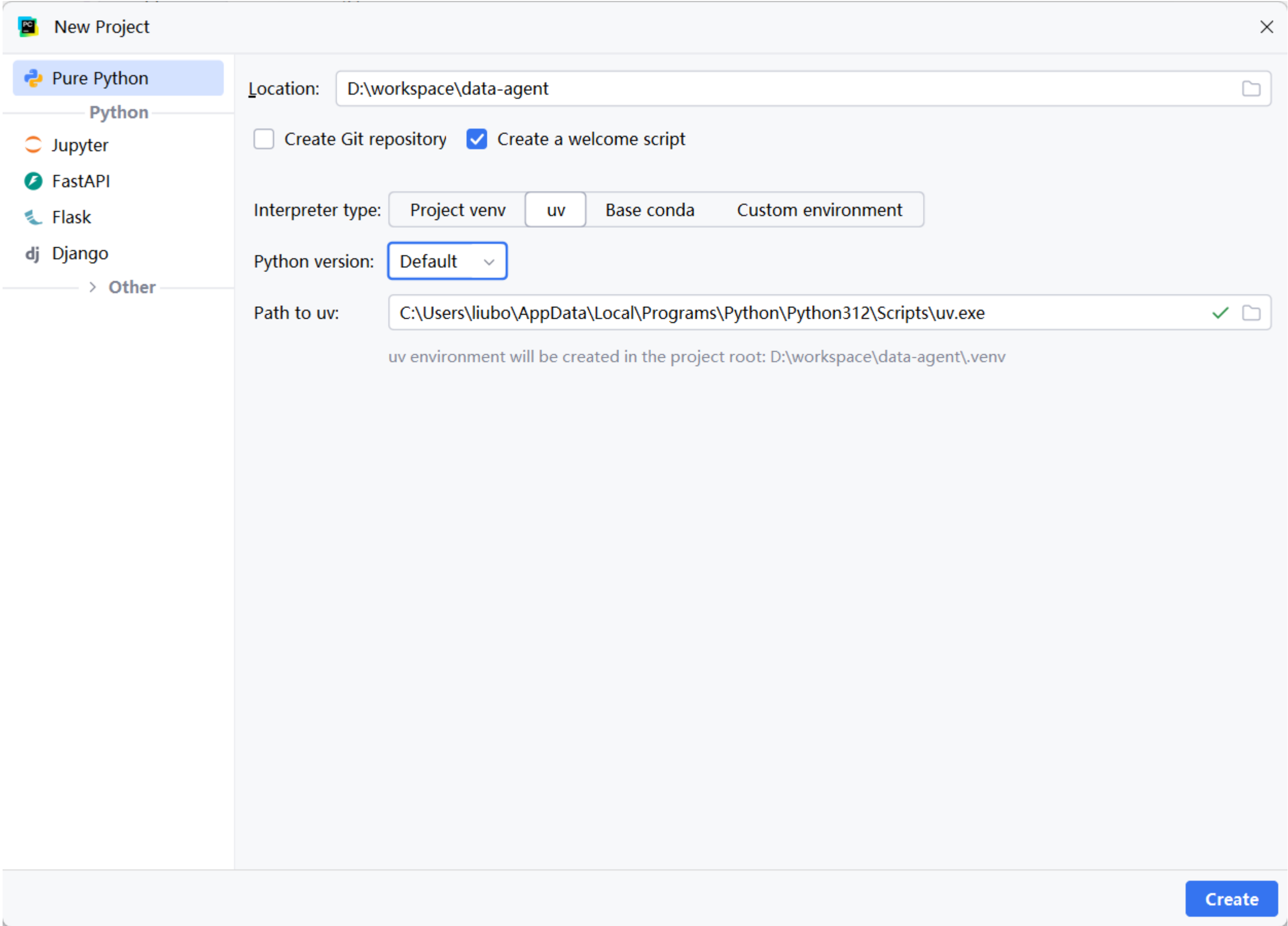
3. 项目开发环境

3.1 项目目录结构

1	data-agent 根目录
2	├─ app 代码目录
3	│ ├─ agent 智能体
4	│ ├─ api 接口
5	│ ├─ clients 数据库客户端
6	│ ├─ config 配置类
7	│ ├─ core 基础设施
8	│ ├─ models 数据库实体类
9	│ ├─ prompt 提示词工具
10	│ ├─ repositories Repo层--负责数据库底层查询
11	│ ├─ schemas 接口数据实体
12	│ ├─ scripts 脚本
13	│ └─ service Service层--负责具体的业务逻辑
14	├─ conf 配置文件
15	├─ docker 开发环境
16	│ ├─ elasticsearch
17	│ ├─ embedding
18	│ └─ mysql
19	├─ logs 日志目录
20	└─ prompts 提示词目录

3.2 创建项目

本项目使用 uv 进行依赖管理与虚拟环境管理，如下图所示



3.3 安装项目所需依赖

以下是项目所需的全部依赖

1	uv add asyncmy cryptography "elasticsearch[async]>=8,<9" fastapi[standard] huggingface-hub jieba langchain langchain-deepseek langchain-huggingface langgraph loguru omegaconf pyyaml qdrant-client sqlalchemy
---	--

3.4 搭建开发环境

本项目采用 Docker 管理开发环境，相关容器配置文件已提供于课程资料中的 docker 目录。将该目录拷贝至项目根目录后，进入 docker 目录并执行 docker compose up -d，即可一键启动项目所需的全部基础服务。

▼	●	data-agent	-	-	-
	●	qdrant	cb4dde91ae47	qdrant/qdrant:v1.16	6333:6333 ↗ Show all ports (2)
	●	elasticsearch	4d6d8a39c656	data-agent-elasticsearch	9200:9200 ↗
	●	embedding	049f6d7558da	huggingface/text-embeddings-inference:cpu-1.	8081:80 ↗
	●	mysql	82cf25b7acb3	mysql:8.0	3306:3306 ↗
	●	kibana	3f1521b3f86c	kibana:8.19.10	5601:5601 ↗

4. 项目基础设施

4.1 配置参数管理

4.1.1 配置文件

本项目采用 YAML 文件管理配置参数，配置文件的路径为 data-agent/conf/app_config.yaml。以下是本项目所需的全部参数。

```
1 logging:
2   file:
3     enable: true
4     level: INFO
5     path: logs
6     rotation: "10 MB"
7     retention: "7 days"
8   console:
9     enable: true
10    level: INFO
11
12 db_meta:
13   host: localhost
14   port: 3306
15   user: atguigu
16   password: Atguigu.123
17   database: meta
18
19 db_dw:
20   host: localhost
21   port: 3306
22   user: atguigu
23   password: Atguigu.123
24   database: dw
25
26 qdrant:
27   host: localhost
28   port: 6333
29   embedding_size: 1024
30
31 embedding:
32   host: localhost
33   port: 8081
34   model: BAAI/bge-large-zh-v1.5
35
36
37 es:
38   host: localhost
39   port: 9200
40   index_name: data_agent
41
42
43 llm:
44   model_name: deepseek-chat
45   api_key: <deepseek_api_key>
```

4.1.2 加载工具

本项目使用的yaml配置文件加载工具为[OmegaConf](#)，具体用法参考其官网即可。

用于读取配置文件的代码放置在 `data-agent/app/conf/app_config.py` 文件中，具体内容如下：

```
1 from dataclasses import dataclass
2 from pathlib import Path
3
4 from omegaconf import OmegaConf
5
6
7 # 日志配置
8 @dataclass
9 class File:
10     enable: bool
11     level: str
12     path: str
13     rotation: str
14     retention: str
15
16
17 @dataclass
18 class Console:
19     enable: bool
20     level: str
21
22
23 @dataclass
24 class LoggingConfig:
25     file: File
26     console: Console
27
28
29 # 数据库配置
30 @dataclass
31 class DBConfig:
32     host: str
33     port: int
34     user: str
35     password: str
36     database: str
37
38
39 @dataclass
40 class QdrantConfig:
41     host: str
42     port: int
43     embedding_size: int
44
45
46 @dataclass
47 class EmbeddingConfig:
48     host: str
49     port: int
50     model: str
51
52
53 @dataclass
54 class ESConfig:
55     host: str
56     port: int
57     index_name: str
58
59
60 @dataclass
61 class LLMConfig:
```

```
62     model_name: str
63     api_key: str
64
65
66 @dataclass
67 class AppConfig:
68     logging: LoggingConfig
69     db_meta: DBConfig
70     db_dw: DBConfig
71     qdrant: QdrantConfig
72     embedding: EmbeddingConfig
73     es: ESConfig
74     llm: LLMConfig
75
76
77 config_file = Path(__file__).parents[2] / 'conf' / 'app_config.yaml'
78 context = OmegaConf.load(config_file)
79 schema = OmegaConf.structured(AppConfig)
80 app_config: AppConfig = OmegaConf.to_object(OmegaConf.merge(schema, context))
```

4.2 MySQL库客户端管理

本项目中的MySQL客户端使用[SQLAlchemy](#)，具体用法参考官方文档。

在data-agent/app/clients/mysql_client.py中编写如下代码，用来管理MySQL客户端。

```
1  import asyncio
2
3  from sqlalchemy import text
4  from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker
5
6  from app.conf.app_config import DBConfig, app_config
7
8
9  class MySQLClientManager:
10     def __init__(self, db_config: DBConfig):
11         self.db_config = db_config
12         self.engine = None
13         self.session_factory = None
14
15     def _get_url(self):
16         return f"mysql+asyncmy://{self.db_config.user}:{self.db_config.password}@{self.db_config.host}:{self.db_config.port}/{self.db_config.database}?charset=utf8mb4"
17
18     def init(self):
19         self.engine = create_async_engine(self._get_url())
20         self.session_factory = async_sessionmaker(self.engine)
21
22     def close(self):
23         if self.engine:
24             self.engine.dispose()
25
26
27 dw_mysql_client_manager = MySQLClientManager(app_config.db_dw)
28 meta_mysql_client_manager = MySQLClientManager(app_config.db_meta)
29
30 if __name__ == '__main__':
31     dw_mysql_client_manager.init()
32
33
34     async def test():
35         async with dw_mysql_client_manager.session_factory() as session:
36             result = await session.execute(text("show tables;"))
37             print(result.fetchall())
38
39
40     asyncio.run(test())
```

4.3 Qdrant客户端管理

本项目中的Qdrant客户端使用[qdrant-client](#)，具体用法参考官方文档。

在data-agent/app/clients/qdrant_client.py中编写如下代码，用来管理Qdrant客户端。

```
1 import asyncio
2 import random
3
4 from qdrant_client import AsyncQdrantClient, models
5
6 from app.conf.app_config import QdrantConfig, app_config
7
8
9 class QdrantClientManager:
10     def __init__(self, qdrant_config: QdrantConfig):
11         self.qdrant_config = qdrant_config
12         self.client: AsyncQdrantClient = None
13
14     def _get_url(self):
15         return f"http://{self.qdrant_config.host}:{self.qdrant_config.port}"
16
17     def init(self):
18         self.client = AsyncQdrantClient(self._get_url())
19
20     async def close(self):
21         await self.client.close()
22
23
24 qdrant_client_manager = QdrantClientManager(app_config.qdrant)
```

4.4 ES客户端管理

本项目中的ES客户端使用[elasticsearch](#)，具体用法参考官方文档。

在data-agent/app/clients/es_client.py中编写如下代码，用来管理ES客户端。

```
1 from elasticsearch import AsyncElasticsearch
2
3 from app.config.app_config import ESConfig, app_config
4
5
6 class ESClientManager:
7     def __init__(self, config: ESConfig):
8         self.config = config
9         self.client: AsyncElasticsearch | None = None
10
11     def _get_url(self):
12         return f"http://{self.config.host}:{self.config.port}"
13
14     def init(self):
15         self.client = AsyncElasticsearch(hosts=[self._get_url()])
16
17     async def close(self):
18         await self.client.close()
19
20
21 es_client_manager = ESClientManager(app_config.es)
```

4.5 日志管理

本项目使用[loguru](#)管理日志，具体用法参考官网。

在data-agent/app/core/logging.py中编写如下代码，统一管理日志。

```
1 import sys
2 import uuid
3 from pathlib import Path
4
5 from loguru import logger
```

```
6
7 from app.config.app_config import app_config
8 from app.core.context import request_id_ctx_var
9
10 log_format = (
11     "<green>{time:YYYY-MM-DD HH:mm:ss.SSS}</green> | "
12     "<level>{level: <8}</level> | "
13     "<magenta>request_id - {extra[request_id]}</magenta> | "
14     "<cyan>{name}</cyan>:<cyan>{function}</cyan>:<cyan>{line}</cyan> - "
15     "<level>{message}</level>"
16 )
17
18
19 def inject_request_id(record):
20     try:
21         request_id = request_id_ctx_var.get()
22     except Exception as e:
23         request_id = uuid.uuid4()
24     record["extra"]["request_id"] = request_id
25
26
27 logger.remove()
28 logger = logger.patch(inject_request_id)
29 if app_config.logging.console.enable:
30     logger.add(sink=sys.stdout, level=app_config.logging.console.level, format=log_format)
31 if app_config.logging.file.enable:
32     path = Path(app_config.logging.file.path)
33     path.mkdir(parents=True, exist_ok=True)
34     logger.add(
35         sink=path / "app.log",
36         level=app_config.logging.file.level,
37         format=log_format,
38         rotation=app_config.logging.file.rotation,
39         retention=app_config.logging.file.retention,
40         encoding="utf-8"
41     )
42
43 if __name__ == '__main__':
44     logger.info("hello world")
45
```

5. 构建元数据知识库

具体实现参考课程资料中的代码即可。

6. 构建问数智能体

具体实现参考课程资料中的代码即可。

7. 前后端联调

具体实现参考课程资料中的代码即可。